SQL Notes and Solutions

Authored by : Arpit Raj , Lnmiit jaipur

1. Sales in First Quarter

Tables

Product

Column Name	Type
product_id	int
product_name	varchar
unit_price	int

Sales

Column Name	Type
seller_id	int
product_id	int
buyer_id	int
sale_date	date
quantity	int
price	int

> Find products that were sold only in the first quarter of 2019:
>
> • Between 2019-01-01 and 2019-03-31 (inclusive).
> • They must not have any sales outside this range.
>
> **Return:** product_id, product_name

Solution 1 — EXISTS / NOT EXISTS

```
SELECT p.product_id,p.product_name
FROM Product p
WHERE EXISTS(

    SELECT 1
    FROM Sales s

    WHERE p.product_id=s.product_id AND
    sale_date BETWEEN '2019-01-01' AND '2019-03-31'
)

AND

NOT EXISTS(
    SELECT 1
    FROM Sales s
    WHERE p.product_id=s.product_id AND
    sale_date NOT BETWEEN '2019-01-01' AND '2019-03-31'
);
```

Solution 2 — Approach 2 — GROUP BY + HAVING

```
SELECT
    p.product_id,
    p.product_name
FROM Product p
JOIN Sales s
ON p.product_id = s.product_id
GROUP BY p.product_id, p.product_name
HAVING
    MIN(s.sale_date) >= '2019-01-01'
    AND
    MAX(s.sale_date) <= '2019-03-31';
```

2. Valid Emails

> Write a solution to find the users who have valid emails.

A valid e-mail has a prefix name and a domain where:

- The prefix name is a string that may contain letters (upper or lower case), digits, underscore '_', period '.', and/or dash '-'. The prefix name must start with a letter.
- The domain must be exactly '@leetcode.com' in lowercase.

Return the result table in any order.

Solution

```
SELECT *
FROM Users
WHERE REGEXP_LIKE (mail,'^[A-Za-z][A-Za-z0-9_.-]*@leetcode\.com$','c');
```

3. Product Orders in Period

Tables

Products

Column Name	Type
product_id	int
product_name	varchar
product_category	varchar

Orders

Column Name	Type
product_id	int
order_date	date
unit	int

Find products that satisfy:

- Ordered during February 2020 (2020-02-01 to 2020-02-29).

- Total units ordered in February ≥ 100.

Return: product_name, unit

Solution

```
# Write your MySQL query statement below
SELECT p.product_name,SUM(o.unit) AS unit
FROM Products p
JOIN Orders o
ON p.product_id=o.product_id

WHERE o.order_date BETWEEN '2020-02-01' AND '2020-02-29'
GROUP BY p.product_id
HAVING SUM(o.unit)>=100;
```

4. Stock Capital Gain/Loss

Table: Stocks

Column Name	Type
stock_name	varchar
operation	enum('Buy', 'Sell')
operation_day	int
price	int

For each stock, calculate its capital gain or loss.

- Buy → money spent (subtract)
- Sell → money earned (add)

Return: stock_name, capital_gain_loss

Solution

```
# Write your MySQL query statement below
SELECT stock_name,SUM(

    CASE WHEN operation='buy' THEN -price
    ELSE price

    END

) AS capital_gain_loss

FROM Stocks
GROUP BY stock_name;
```

5. Unique Subjects Taught

Table: Teacher

Column Name	Type
teacher_id	int
subject_id	int
dept_id	int

> Write a solution to calculate the number of unique subjects each teacher teaches in the university.
>
> Return the result table in any order.

Solution

```
SELECT teacher_id,COUNT(

    DISTINCT subject_id
) AS cnt

FROM Teacher
GROUP BY teacher_id;
```

6. Rising Temperature

Table: Weather

Column Name	Type
id	int
recordDate	date
temperature	int

> Write a solution to find all dates' id with higher temperatures compared to its previous dates (yesterday).
>
> Return the result table in any order.

Solution

```
SELECT w1.id

FROM Weather w1
JOIN Weather w2
ON w1.recordDate=DATE_ADD(w2.recordDate, INTERVAL 1 DAY)

WHERE w1.temperature>w2.temperature;
```

7. Daily Active Users

Table: Activity

Column Name	Type
user_id	int
session_id	int
activity_date	date
activity_type	enum

> Write a solution to find the daily active user count for a period of 30 days ending 2019-07-27 inclusively. A user was active on someday if they made at least one activity on that day.
>
> Return the result table in any order.

Solution

```
# Write your MySQL query statement below
SELECT activity_date AS day,
COUNT(
    DISTINCT(user_id)
) AS active_users

FROM Activity
WHERE activity_date BETWEEN DATE_SUB('2019-07-27', INTERVAL 29 DAY) AND
'2019-07-27'
GROUP BY activity_date;
```

8. Latest Login

Table: Logins

Column Name	Type
user_id	int
time_stamp	datetime

(user_id, time_stamp) is the primary key (combination of columns with unique values) for this table.
Each row contains information about the login time for the user with ID user_id.

Write a solution to report the latest login for all users in the year 2020. Do not include the users who did not login in 2020.

Return the result table in any order.

Solution

```
# Write your MySQL query statement below
SELECT user_id,

MAX(
    time_stamp
) AS last_stamp

FROM Logins
WHERE YEAR(time_stamp)=2020
GROUP BY user_id;
```

9. Missing Information

Table: Employees

Column Name	Type
employee_id	int
name	varchar

Table: Salaries

Column Name	Type
employee_id	int
salary	int

employee_id is the column with unique values for this table.
Each row of this table indicates the name (or salary) of the employee whose ID is employee_id.

Write a solution to report the IDs of all the employees with missing information. The information of an employee is missing if:

- The employee's name is missing, or
- The employee's salary is missing.

Return the result table ordered by employee_id in ascending order.

Solution

```
# Write your MySQL query statement below
SELECT employee_id
FROM Employees e
WHERE NOT EXISTS(
    SELECT 1
    FROM Salaries s
    WHERE e.employee_id=s.employee_id
)

UNION

SELECT employee_id
FROM salaries s
WHERE NOT EXISTS(
    SELECT 1
    FROM Employees e
    WHERE e.employee_id=s.employee_id
)

ORDER BY employee_id;
```

10. Followers Count

Input: Followers Table

user_id	follower_id
0	1
1	0
2	0
2	1

Output

user_id	followers_count
0	1

1	1
2	2

Solution

```
SELECT user_id ,
COUNT(follower_id) AS followers_count

FROM Followers
GROUP BY user_id
ORDER BY user_id;
```